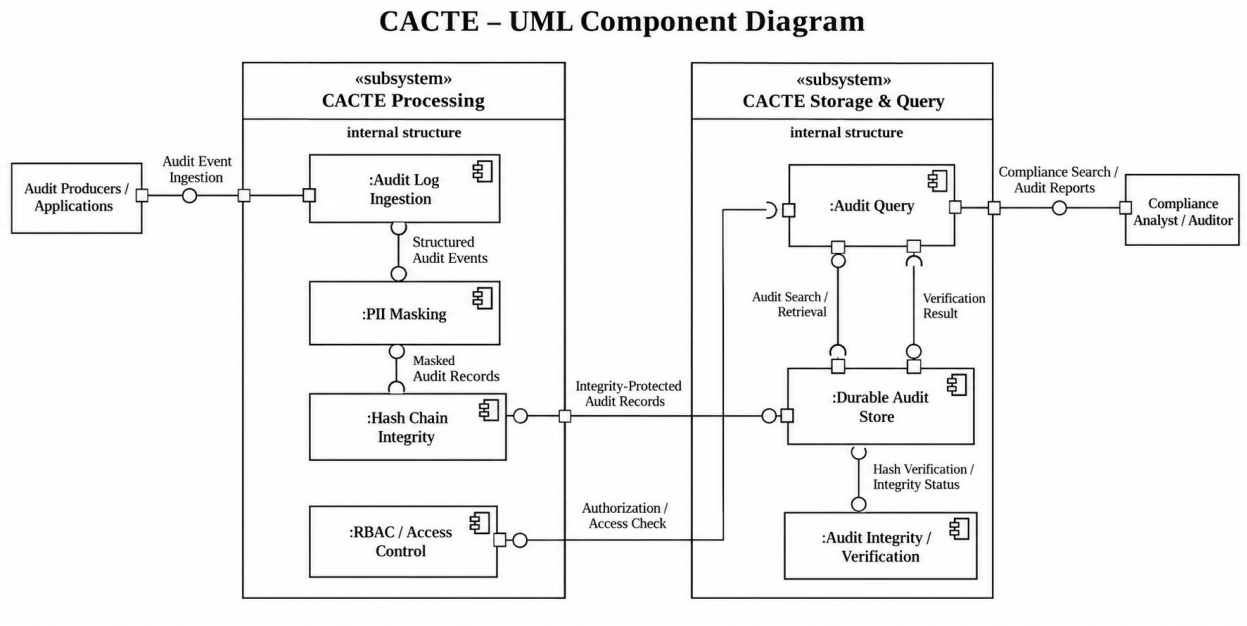
Lab 3: Architectural Pattern Selection & Component Modelling
Centralized Audit Trail Compliance Engine (CATCE)



Architecture Selection: We chose Microservices Architecture for the Centralized Audit Trail Compliance Engine System.

Architectural Choice

The CATCE system is composed of functionally distinct, independently operable concerns: log ingestion and PII masking, hash-chain integrity verification, role-based access control, query performance at scale, and durability through replication. Each concern already maps to a separate epic in the project backlog, reflecting a natural service boundary rather than a layered decomposition. A Microservices Architecture was therefore selected over Layered or Client-Server styles.

Reason 1: Independent Scaling
The Query Service must sustain load-tested performance under simulated peak concurrency, while the Log Ingestion Service handles a comparatively steady, append-only write throughput. These two components have fundamentally different scaling profiles. Microservices allow the Query Service to be scaled and indexed independently without over-provisioning or redeploying the ingestion path.

Reason 2: Fault Isolation
Durability requirements demand that the system tolerate node failure mid-write while preserving 100% of acknowledged records. In a microservices architecture, a failure in the Replication and Durability Service is isolated and does not cascade into the Ingestion, Query, or Access Control services, keeping the audit trail available even during a partial outage.

Security Advantage
Access control is isolated in a dedicated Access Control Service rather than embedded as scattered logic inside other services. Both the Hash Chain Integrity Service and the Query Service delegate authorization checks to this single service before exposing verification results or query data, centralizing the enforcement of roles such as Security Auditor and reducing the risk of an authorization bypass.

Performance Benefit
Because the Query Service is deployed and indexed independently, filtering and benchmarking optimizations (such as index strategy changes for complex queries) can be applied and scaled without impacting the throughput of log ingestion or hash-chain verification, keeping write-path latency unaffected by read-path load.